

PCCST503 – Machine Learning
Assignment 1
Design of a Safe Semantic Planner in a Finite
Cartesian State Space

Department of Computer Science and Engineering

Objective

The objective of this assignment is to design and implement a generic planning algorithm that computes a safe path in a finite Cartesian state space. Although inspired by AI planning, this assignment emphasizes graph search, heuristic design, optimization, software engineering, and experimental evaluation.

Problem Definition

Let

$$S = \{s_1, s_2, \dots, s_n\}$$

be a finite set of states embedded in a Cartesian space

$$\mathbb{R}^d.$$

Each state has a vector representation

$$s_i = (x_1, x_2, \dots, x_d).$$

The planner receives

- Initial state s_I
- Goal state s_G
- Set of bad states

$$B = \{b_1, b_2, \dots, b_k\}$$

- Directed transitions

$$T = \{(s_i, s_j)\}$$

Each transition additionally contains

- Cost

- Reliability
- Safety score
- Availability flag

The planner must compute a sequence of transitions from the initial state to the goal while avoiding bad states.

Optimization Objectives

A valid planner should satisfy:

1. Reach the goal state.
2. Never visit a bad state.
3. Minimize total transition cost.
4. Maximize the minimum Euclidean distance from every visited state to the nearest bad state.
5. Produce the solution within reasonable execution time.

An example objective function is

$$Score(P) = \alpha G - \beta C + \gamma D + \delta R$$

where

- G = goal completion
- C = cumulative transition cost
- D = minimum safety distance
- R = cumulative reliability

Dynamic Environment

The planner should be designed so that the following may change periodically:

- Goal state
- Bad states
- Transition availability
- New transitions
- Removed transitions

Students should explain how their implementation can efficiently replan after such updates.

Suggested Software Interfaces

State

```
class State
{
public:
    uint64_t id;
    std::vector<double> embedding;
};
```

Transition

```
class Transition
{
public:
    uint64_t id;
    uint64_t from;
    uint64_t to;
    double cost;
    double safety;
    double reliability;
    bool available;
};
```

Planning Problem

```
class PlanningProblem
{
public:
    uint64_t initialState;
    uint64_t goalState;
    std::vector<uint64_t> badStates;
    std::vector<State> states;
    std::vector<Transition> transitions;
};
```

Planning Result

```
class PlanningResult
{
public:
    bool success;
    std::vector<uint64_t> statePath;
    std::vector<uint64_t> transitionPath;
    double totalCost;
```

```
    double safetyScore;  
};
```

Planner Interface

```
class Planner  
{  
public:  
    virtual PlanningResult plan(  
        const PlanningProblem& problem)=0;  
};
```

Algorithm Design Guidelines

Students may choose one of the following:

- LPA*
- D* Lite

The report must clearly describe

1. State representation
2. Data structures
3. Heuristic function
4. Safety computation
5. Time complexity
6. Space complexity

Illustrative Test Cases

Test Case 1: Basic Reachability

Graph:

$$S \rightarrow A \rightarrow B \rightarrow G$$

Expected result: planner returns the unique valid path.

Test Case 2: Bad State Avoidance

Paths:

$$S \rightarrow A \rightarrow X \rightarrow G$$

where X is a bad state,
and

$$S \rightarrow C \rightarrow D \rightarrow G.$$

Expected result: second path must be selected.

Test Case 3: Safety Margin

Two valid paths exist.

Path 1 has lower cost but passes close to bad states.

Path 2 has higher cost but remains significantly farther away.

Students should explain how the planner balances cost and safety.

Test Case 4: Dynamic Transition

Initially,

$$S \rightarrow A \rightarrow G$$

Later, transition (A, G) becomes unavailable.

Expected result: planner computes an alternative path.

Test Case 5: Goal Update

The goal changes during execution.

The planner should produce a revised path without rebuilding all data structures whenever possible.

Test Case 6: Transition Addition

A new shortcut transition is inserted.

Expected result: planner discovers the improved solution.

Students should evaluate:

- Goal success rate
- Number of bad states visited (expected zero)
- Total path cost
- Minimum distance to bad states
- Number of explored states
- Planning time

- Memory usage
- Replanning time

Deliverables

1. C++ source code.
2. Design report.
3. Experimental results.
4. User manual.
5. Demonstration.

Bonus (Optional)

Implement one or more of the following:

- Multi-goal planning.
- Time-dependent transition availability.
- Incremental replanning.
- Parallel search.
- Learning-based heuristic.
- Test it on Knowledge graph